

Universidad Autónoma de Tamaulipas

Facultad de Ingeniería Tampico



ASIGNATURA

PROGRAMACION DE INTERFACES Y

PUERTOS

6to. Semestre – Grupo “i”
2025 -1

TRABAJO

Desarrollo de Prácticas y Proyectos

UNIDAD

2 -DESARROLLO E IMPLEMENTACIÓN DE CONTROLADORES MEDIANTE
INTERFACES Y PUERTOS

Docente: Dr. García Ruiz Alejandro H.

Integrante del Equipo	Nivel de Participación
Izaguirre Cortes Emanuel	60%
Turrubiates Mejia Gilberto	20%
García Salas Yahir Misael	20%
Total:	100%

Índice

Índice	1
Repositorio(s) de Prácticas	2
PP1.	2
Descripción de la practica	2
Introducción	2
Componentes para el desarrollo de la practica	2
Desarrollo	3
Pruebas Realizadas	3
Conclusión PP1.....	5
PP2.	6
Descripción de la practica	6
Introducción	6
Componentes para el desarrollo de la practica	6
Desarrollo	6
Pruebas Realizadas	7
Conclusión PP2.....	8
Proyecto_U2.	9
Descripción del proyecto	9
Introducción	9
Componentes para el desarrollo del proyecto	10
Desarrollo	10
Pruebas Realizadas	12
Conclusión de proyecto U2	15

Repositorio(s) de Prácticas

Practica	Repositorio
Practica 1 a la 2 Proyecto_U2	https://github.com/Emanuel-Izaguirre-Cortes-03/Pip_2025_1_eq_4_i/tree/main/Practicas

PP1.

Esta práctica consiste en el desarrollo de una aplicación de escritorio en Python utilizando PyQt5. La aplicación permite mostrar diferentes imágenes según la selección de un usuario mediante un QSlider y verificar si el texto ingresado por el usuario coincide con la imagen mostrada.

Descripción de la practica

Esta práctica consiste en el desarrollo de una aplicación de escritorio en Python utilizando PyQt5. La aplicación permite mostrar diferentes imágenes según la selección de un usuario mediante un QSlider y verificar si el texto ingresado por el usuario coincide con la imagen mostrada.

Introducción

Las interfaces gráficas de usuario (GUI) facilitan la interacción con aplicaciones mediante elementos visuales. PyQt5 es un framework popular para la creación de interfaces en Python. En esta práctica, se desarrolló una aplicación donde el usuario debe identificar correctamente el nombre de una imagen seleccionada aleatoriamente.

Componentes para el desarrollo de la practica

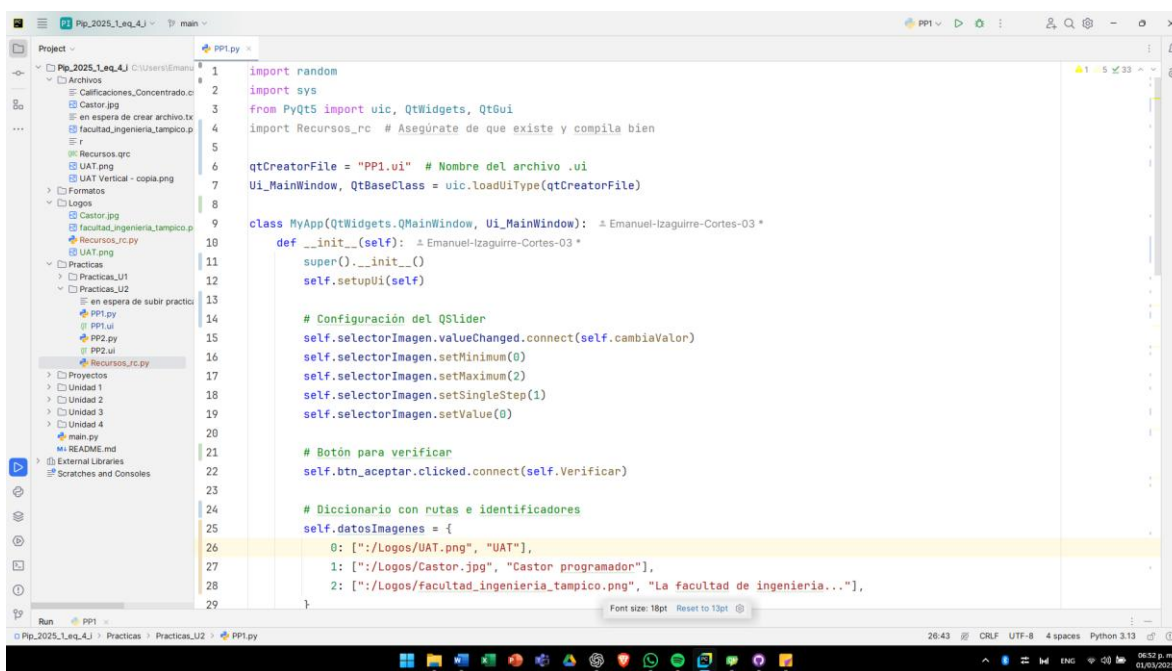
- **Lenguaje de programación:** Python 3
- **Bibliotecas:** PyQt5, random, sys
- **Widgets de PyQt5:** QLabel, QSlider, QPushButton, QMessageBox
- **Archivos:** Archivo .ui para la interfaz diseñada en Qt Designer y un script .py para la lógica de la aplicación

Desarrollo

1. Se diseñó la interfaz en Qt Designer y se guardó en un archivo .ui.
2. Se cargó la interfaz en Python usando `uic.loadUiType()`.
3. Se implementó un diccionario con las rutas de las imágenes y sus nombres.
4. Se configuró un QSlider para cambiar las imágenes según su valor.
5. Se implementó un método para seleccionar aleatoriamente una imagen y mostrar su nombre en un QLineEdit.
6. Se programó la verificación de la coincidencia entre el nombre ingresado y el correcto.

Pruebas Realizadas

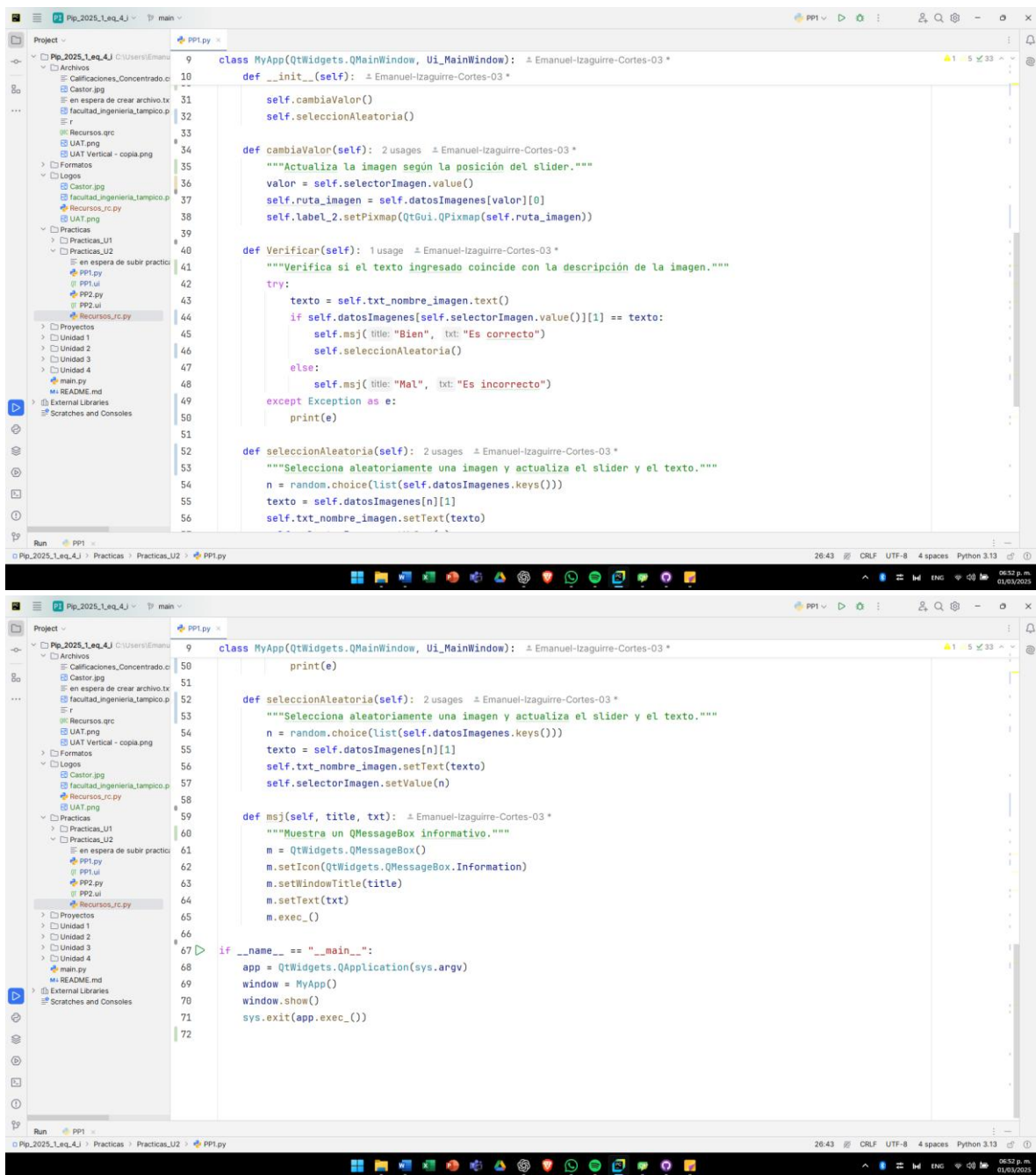
- Se probó la carga de imágenes según la selección del slider.
- Se verificó la correcta selección aleatoria de imágenes.
- Se comprobó la validación de nombres de imágenes con mensajes informativos.
- Se corrigieron errores en la comparación del nombre ingresado.



```

1 import random
2 import sys
3 from PyQt5 import uic, QtWidgets, QtGui
4 import Recursos_rc # Asegúrate de que existe y compila bien
5
6 qtCreatorFile = "PPI.ui" # Nombre del archivo .ui
7 Ui_MainWindow, QtBaseClass = uic.loadUiType(qtCreatorFile)
8
9 class MyApp(QtWidgets.QMainWindow, Ui_MainWindow): # Emanuel-Izaguirre-Cortes-03 *
10     def __init__(self): # Emanuel-Izaguirre-Cortes-03 *
11         super().__init__()
12         self.setupUi(self)
13
14         # Configuración del QSlider
15         self.selectorImagen.valueChanged.connect(self.cambiaValor)
16         self.selectorImagen.setMinimum(0)
17         self.selectorImagen.setMaximum(2)
18         self.selectorImagen.setSingleStep(1)
19         self.selectorImagen.setValue(0)
20
21         # Botón para verificar
22         self.btn_aceptar.clicked.connect(self.Verificar)
23
24         # Diccionario con rutas e identificadores
25         self.datosImagenes = {
26             0: ["/Logos/UAT.png", "UAT"],
27             1: ["/Logos/Castor.jpg", "Castor programador"],
28             2: ["/Logos/facultad_ingenieria_tampico.png", "La facultad de ingeniería..."],
29         }

```



```

class MyApp(QMainWindow, Ui_MainWindow):
    def __init__(self):
        self.cambiaValor()
        self.seleccionAleatoria()

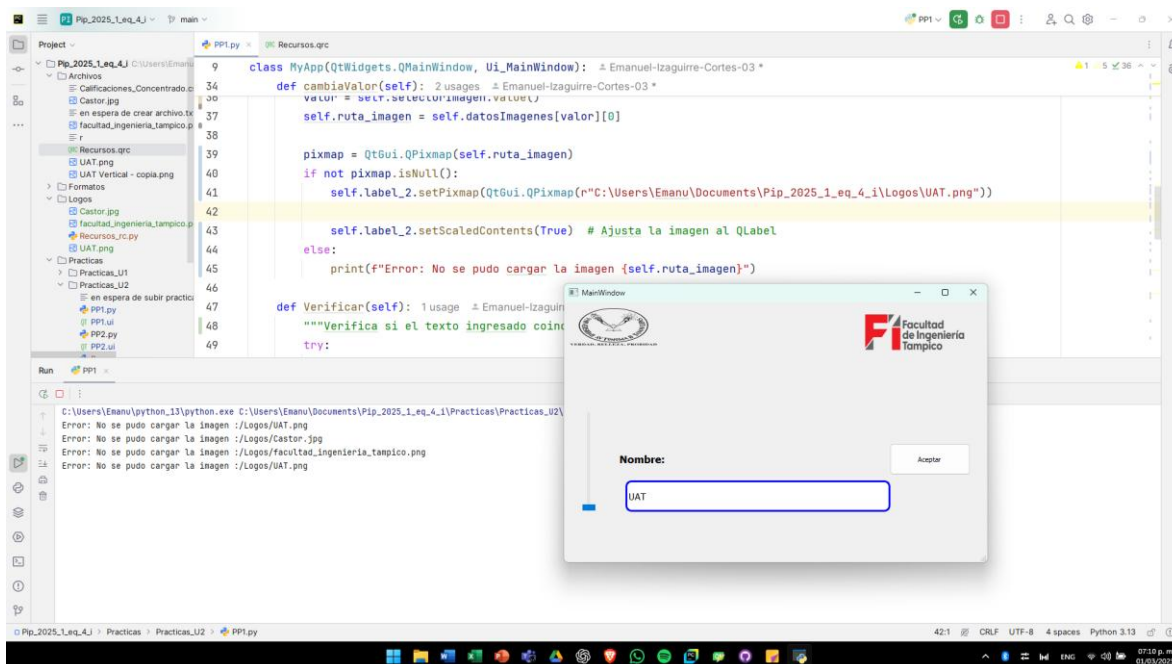
    def cambiaValor(self):
        """Actualiza la imagen según la posición del slider."""
        valor = self.selectorImagen.value()
        self.ruta_imagen = self.datosImagenes[valor][0]
        self.label_2.setPixmap(QtGui.QPixmap(self.ruta_imagen))

    def Verificar(self):
        """Verifica si el texto ingresado coincide con la descripción de la imagen."""
        try:
            texto = self.txt_nombre_imagen.text()
            if self.datosImagenes[self.selectorImagen.value()][1] == texto:
                self.msj(title="Bien", txt="Es correcto")
                self.seleccionAleatoria()
            else:
                self.msj(title="Mal", txt="Es incorrecto")
        except Exception as e:
            print(e)

    def seleccionAleatoria(self):
        """Selecciona aleatoriamente una imagen y actualiza el slider y el texto."""
        n = random.choice(list(self.datosImagenes.keys()))
        texto = self.datosImagenes[n][1]
        self.txt_nombre_imagen.setText(texto)

    def msj(self, title, txt):
        """Muestra un QMessageBox informativo."""
        m = QtWidgets.QMessageBox()
        m.setIcon(QtWidgets.QMessageBox.Information)
        m.setWindowTitle(title)
        m.setText(txt)
        m.exec_()

if __name__ == "__main__":
    app = QtWidgets.QApplication(sys.argv)
    window = MyApp()
    window.show()
    sys.exit(app.exec_())
  
```



Conclusión PP1

En esta práctica, se implementó una interfaz gráfica utilizando **PyQt5**, permitiendo la manipulación de imágenes a través de un **QSlider** y verificando la correspondencia entre el nombre ingresado y la imagen mostrada.

Al concluir el desarrollo, se identificaron varios aspectos clave:

1. **Manejo de Archivos UI:** La correcta carga del archivo .ui con `uic.loadUiType()` facilita la integración de diseños creados en Qt Designer, reduciendo el tiempo de desarrollo.
2. **Gestión de Rutas de Imágenes:** Se evidenció la importancia de definir rutas de imágenes correctamente, evitando errores relacionados con secuencias de escape (`\U`) en Windows.
3. **Interactividad con el Usuario:** La validación de texto y el uso de `QMessageBox` para mostrar mensajes de acierto o error mejoraron la experiencia del usuario, permitiendo una retroalimentación inmediata.
4. **Aleatorización de Datos:** Se utilizó la función `random.choice()` para seleccionar imágenes de manera aleatoria, lo que diversificó la interacción con el programa y reforzó conceptos de estructuras de datos como diccionarios en Python.
5. **Importancia del Debugging:** Se resolvieron errores de sintaxis y se optimizó la gestión de eventos, resaltando la necesidad de pruebas constantes para garantizar un funcionamiento correcto.

En general, la práctica permitió reforzar conocimientos en **interfaces gráficas, manipulación de imágenes, eventos en GUI y validación de datos**, habilidades esenciales para el desarrollo de software interactivo.

PP2.

Descripción de la practica

La aplicación permite iniciar un temporizador que se actualiza cada segundo y muestra el tiempo transcurrido en una etiqueta. Se ha desarrollado utilizando PyQt5 y un diseño de interfaz definido en un archivo de código Python.

Introducción

Este proyecto es una aplicación en Python con una interfaz gráfica (GUI) desarrollada con PyQt5. La aplicación implementa un temporizador que muestra el tiempo transcurrido en segundos.

Componentes para el desarrollo de la practica

Lenguaje de programación**: Python

- **Bibliotecas**:

- PyQt5 (para la interfaz gráfica)
- QtCore (para el temporizador)
- QtWidgets (para los elementos de la interfaz)

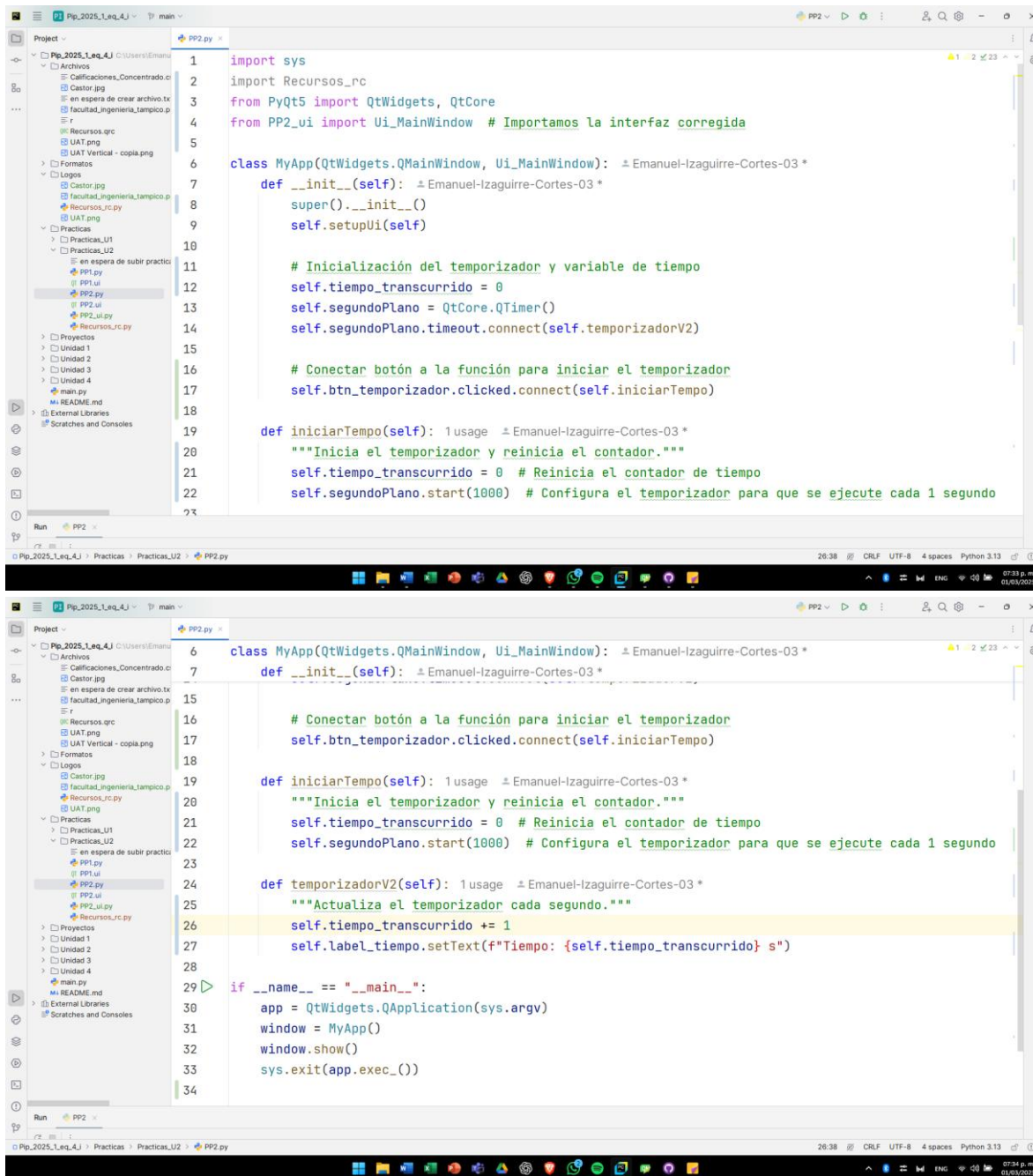
- **Archivos**:

- `PP2\_ui.py`: Define la interfaz gráfica.
- `PP2.py`: Contiene la lógica del programa y la interacción con la interfaz.

Desarrollo

1. Se creó la interfaz gráfica en `PP2\_ui.py`, definiendo un botón (`btn\_temporizador`) y una etiqueta (`label\_tiempo`).
2. Se implementó la lógica en `PP2.py`, donde:
 - Se inicializa la interfaz y los componentes.
 - Se configura un temporizador que se actualiza cada segundo.
 - Se maneja el evento del botón para iniciar el temporizador.

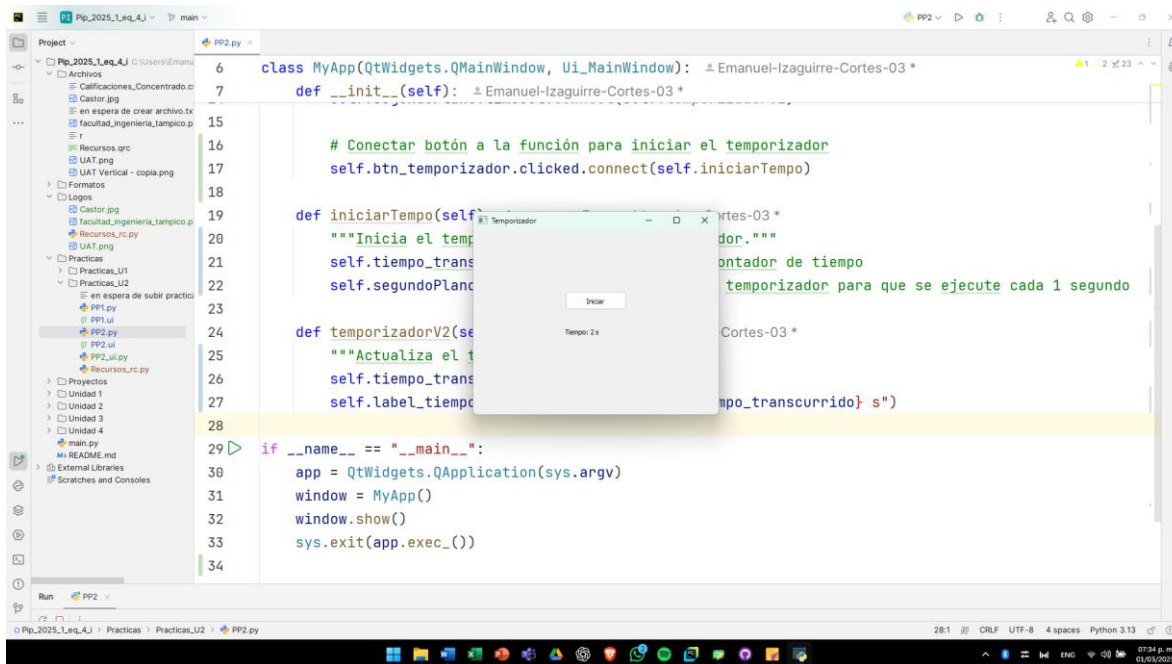
Pruebas Realizadas



```

1 import sys
2 import Recursos_rc
3 from PyQt5 import QtWidgets, QtCore
4 from PP2_ui import Ui_MainWindow # Importamos la interfaz corregida
5
6 class MyApp(QtWidgets.QMainWindow, Ui_MainWindow): # Emanuel-Izaguirre-Cortes-03 *
7     def __init__(self): # Emanuel-Izaguirre-Cortes-03 *
8         super().__init__()
9         self.setupUi(self)
10
11         # Inicialización del temporizador y variable de tiempo
12         self.tiempo_transcurrido = 0
13         self.segundoPlano = QtCore.QTimer()
14         self.segundoPlano.timeout.connect(self.temporizadorV2)
15
16         # Conectar botón a la función para iniciar el temporizador
17         self.btn_temporizador.clicked.connect(self.iniciarTempo)
18
19     def iniciarTempo(self): 1 usage # Emanuel-Izaguirre-Cortes-03 *
20         """Inicia el temporizador y reinicia el contador."""
21         self.tiempo_transcurrido = 0 # Reinicia el contador de tiempo
22         self.segundoPlano.start(1000) # Configura el temporizador para que se ejecute cada 1 segundo
23
24 class MyApp(QtWidgets.QMainWindow, Ui_MainWindow): # Emanuel-Izaguirre-Cortes-03 *
25     def __init__(self): # Emanuel-Izaguirre-Cortes-03 *
26
27         # Conectar botón a la función para iniciar el temporizador
28         self.btn_temporizador.clicked.connect(self.iniciarTempo)
29
30     def iniciarTempo(self): 1 usage # Emanuel-Izaguirre-Cortes-03 *
31         """Inicia el temporizador y reinicia el contador."""
32         self.tiempo_transcurrido = 0 # Reinicia el contador de tiempo
33         self.segundoPlano.start(1000) # Configura el temporizador para que se ejecute cada 1 segundo
34
35     def temporizadorV2(self): 1 usage # Emanuel-Izaguirre-Cortes-03 *
36         """Actualiza el temporizador cada segundo."""
37         self.tiempo_transcurrido += 1
38         self.label_tiempo.setText(f"Tiempo: {self.tiempo_transcurrido} s")
39
40 if __name__ == "__main__":
41     app = QtWidgets.QApplication(sys.argv)
42     window = MyApp()
43     window.show()
44     sys.exit(app.exec_())

```

```

6 class MyApp(QMainWindow, Ui_MainWindow):
7     def __init__(self):
15
16     # Conectar botón a la función para iniciar el temporizador
17     self.btn_temporizador.clicked.connect(self.iniciarTempo)
18
19     def iniciarTempo(self):
20         """Inicia el temporizador"""
21         self.tiempo_transcurrido = 0
22         self.segundoPlanear = QTimer()
23         self.segundoPlanear.timeout.connect(self.actualizarTiempo)
24         self.segundoPlanear.start(1000)
25
26     def actualizarTiempo(self):
27         """Actualiza el tiempo"""
28         self.tiempo_transcurrido += 1
29         self.label_tiempo.setText(f"Tiempo: {self.tiempo_transcurrido} s")
30
31     if __name__ == "__main__":
32         app = QApplication(sys.argv)
33         window = MyApp()
34         window.show()
35         sys.exit(app.exec_())

```

Conclusión PP2

El proyecto cumple con su objetivo de implementar un temporizador simple con una interfaz gráfica en PyQt5. Se logró una integración funcional entre la lógica del programa y la interfaz, permitiendo futuras mejoras o expansión.

Proyecto_U2.

Descripción del proyecto

Este proyecto consiste en el desarrollo de un juego interactivo tipo "**Simón Dice**", implementado en **Python** utilizando **Qt Designer** para la creación de la interfaz gráfica de usuario (**GUI**). El objetivo del juego es que el usuario repita secuencias de colores y sonidos generadas aleatoriamente por el sistema, aumentando progresivamente la dificultad.

El desarrollo del proyecto se basa en los siguientes elementos clave:

- **Interfaz Gráfica:** Creada con **Qt Designer**, manteniendo el archivo en formato .ui sin convertirlo a código Python.
- **Lógica del Juego:** Programada en **Python**, generando secuencias aleatorias de colores que el jugador debe memorizar y repetir correctamente.
- **Interacción con el Usuario:** Uso de eventos de clic para seleccionar los colores en el orden correcto.
- **Retroalimentación Visual y Sonora:** Implementación de efectos de iluminación y sonidos para mejorar la experiencia del usuario.
- **Dificultad Progresiva:** Cada ronda agrega un nuevo color a la secuencia, aumentando el desafío.

El propósito de este proyecto es combinar habilidades de programación con Python, diseño de interfaces gráficas y manipulación de eventos en Qt. Además, permite explorar el desarrollo de juegos educativos y de memoria.

Introducción

Los juegos de memoria y secuencias han sido una herramienta efectiva para mejorar la concentración, la retención y la agilidad mental. Uno de los más populares es "**Simón Dice**", un juego clásico en el que el jugador debe repetir secuencias de colores y sonidos generadas por el sistema.

Este proyecto tiene como objetivo desarrollar una versión digital del juego "**Simón Dice**" utilizando **Python** para la lógica de programación y **Qt Designer** para el diseño de la interfaz gráfica. A diferencia de otras implementaciones, la interfaz se mantendrá en formato .ui sin ser convertida a código Python, permitiendo una estructura modular y flexible.

El juego desafiará al usuario a memorizar y repetir secuencias de colores que se irán volviendo progresivamente más complejas. A través de una combinación de elementos visuales y auditivos, el proyecto busca ofrecer una experiencia interactiva y entretenida, al mismo tiempo que refuerza habilidades cognitivas en los jugadores.

Componentes para el desarrollo del proyecto

1. Software y Herramientas

- **Python:** Lenguaje de programación principal para la lógica del juego.
- **Qt Designer:** Herramienta para diseñar la interfaz gráfica en formato .ui.
- **PyQt:** Biblioteca para conectar la interfaz de Qt Designer con Python.

2. Componentes de la Interfaz Gráfica (GUI)

- **QPushButton:** Botones de colores utilizados como entrada del jugador.
- **QLabel:** Etiquetas para mostrar instrucciones y puntajes.
- **QFrame / QWidget:** Contenedores para organizar los elementos gráficos.
- **QMessageBox:** Ventanas emergentes para notificaciones como "Juego terminado" o "Nivel superado".

3. Funcionalidades y Lógica del Juego

- **Generación de Secuencias Aleatorias:** Uso del módulo random para crear combinaciones de colores.
- **Temporización:** Uso de QTimer para manejar los intervalos entre los colores mostrados.
- **Eventos y Señales:** Uso de clicked.connect() para detectar la selección de colores por el usuario.
- **Sonidos:** Implementación de efectos auditivos con QSound para cada color presionado.
- **Sistema de Puntuación:** Registro de niveles superados y puntajes acumulados.

Estos componentes permiten que el juego sea interactivo, dinámico y funcional, brindando una experiencia de usuario entretenida y desafiante.

Desarrollo

El desarrollo del juego "Simón Dice" se ha dividido en varias etapas, que abarcan desde el diseño de la interfaz hasta la implementación de la lógica del juego. A continuación, se describen las principales fases y cómo se implementaron los diferentes aspectos del proyecto.

1. Diseño de la Interfaz Gráfica (GUI)

El primer paso fue el diseño de la interfaz gráfica utilizando **Qt Designer**. Se optó por un enfoque sencillo y funcional, en el que se incluyen los siguientes elementos:

- **Botones de Colores:** Se utilizaron **QPushButton** para representar los colores que el jugador debe pulsar en el orden correcto. Cada botón tiene un color único, lo que facilita su identificación durante el juego.

- **Pantalla de Instrucciones:** Una **QLabel** fue añadida para proporcionar información al usuario sobre cómo jugar y las reglas básicas.
- **Área de Puntajes:** También se incluyó un área donde se muestra el puntaje actual y el nivel alcanzado. Esta información se actualiza a medida que el jugador avanza en el juego.
- **Botón de Reinicio:** Un **QPushButton** adicional fue agregado para permitir al usuario reiniciar el juego en cualquier momento.

La interfaz fue diseñada en formato **.ui**, lo que permite mantenerla separada del código y facilitar futuras modificaciones.

2. Implementación de la Lógica del Juego

Una vez que la interfaz gráfica fue diseñada, se implementó la lógica del juego en **Python**. Los principales componentes de la lógica son:

- **Generación de Secuencias Aleatorias:** Utilizando el módulo **random**, se creó una función que genera secuencias de colores aleatorios. Estas secuencias se muestran al jugador para que las memorice y las repita.
- **Verificación de Respuestas:** Cuando el jugador intenta repetir la secuencia, el programa verifica si la respuesta es correcta comparando el orden de los colores seleccionados con la secuencia original.
- **Dificultad Progresiva:** A medida que el jugador acierta la secuencia, el juego aumenta la dificultad agregando un color más a la secuencia. Esto hace que el juego sea más desafiante conforme se avanza.
- **Temporización y Control de Intervalos:** Para mostrar cada color de la secuencia al jugador, se utilizó **QTimer** para establecer los intervalos entre cada color. Esto permite que el jugador tenga tiempo suficiente para memorizar cada color antes de que se muestre el siguiente.

3. Interacción con el Usuario

La interacción con el usuario se logra mediante eventos. Cuando el jugador hace clic en uno de los botones de color, el evento de clic es capturado y procesado. Para esto, se utilizó el método **clicked.connect()** de PyQt, que vincula los botones de color con las funciones de verificación y retroalimentación del juego.

4. Retroalimentación Visual y Sonora

Para hacer el juego más interactivo y entretenido, se incorporaron efectos visuales y sonoros:

- **Efectos Visuales:** Los botones de color se iluminan temporalmente cuando se muestran en la secuencia o cuando el jugador hace clic en ellos para repetir la secuencia.

- **Efectos Sonoros:** Se implementaron sonidos correspondientes a cada color para reforzar la experiencia del jugador. Cada vez que un color es mostrado o seleccionado, se reproduce un sonido asociado utilizando **QSound**.

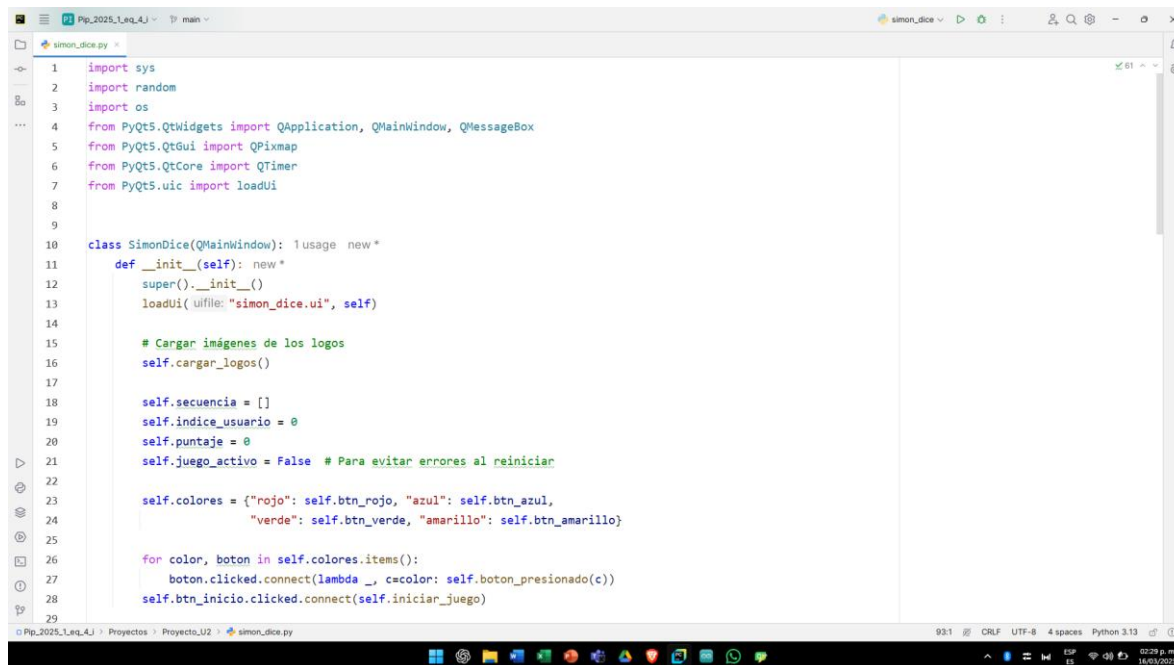
5. Sistema de Puntuación

El sistema de puntuación sigue un formato sencillo: el jugador acumula puntos al acertar correctamente las secuencias. Además, el nivel aumenta a medida que el jugador supera cada ronda, con una nueva secuencia que incluye un color adicional. La puntuación y el nivel se muestran en tiempo real en la interfaz, proporcionando al jugador una indicación clara de su progreso.

Pruebas Realizadas

Después de la implementación, se realizaron pruebas para asegurarse de que todas las funcionalidades estuvieran trabajando correctamente. Se ajustaron los tiempos de los **QTimer**, la dificultad del juego y se optimizó la interfaz para garantizar una experiencia de usuario fluida y atractiva.

Este desarrollo se centró en proporcionar una experiencia interactiva y desafiante para el jugador, utilizando las herramientas y tecnologías disponibles de manera eficiente. La combinación de una interfaz gráfica atractiva y una lógica de juego progresiva asegura que el proyecto sea entretenido y educativo al mismo tiempo.



```

1  import sys
2  import random
3  import os
4  from PyQt5.QtWidgets import QApplication, QMainWindow, QMessageBox
5  from PyQt5.QtGui import QPixmap
6  from PyQt5.QtCore import QTimer
7  from PyQt5.uic import loadUi
8
9
10 class SimonDice(QMainWindow):
11     def __init__(self):
12         super().__init__()
13         loadUi(os.path.join("ui", "simon_dice.ui"), self)
14
15         # Cargar imágenes de los logos
16         self.cargar_logos()
17
18         self.secuencia = []
19         self.indice_usuario = 0
20         self.puntaje = 0
21         self.juego_activo = False # Para evitar errores al reiniciar
22
23         self.colores = {"rojo": self.btn_rojo, "azul": self.btn_azul,
24                        "verde": self.btn_verde, "amarillo": self.btn_amarillo}
25
26         for color, boton in self.colores.items():
27             boton.clicked.connect(lambda _, c=color: self.boton_presionado(c))
28         self.btn_inicio.clicked.connect(self.iniciar_juego)
29

```

```

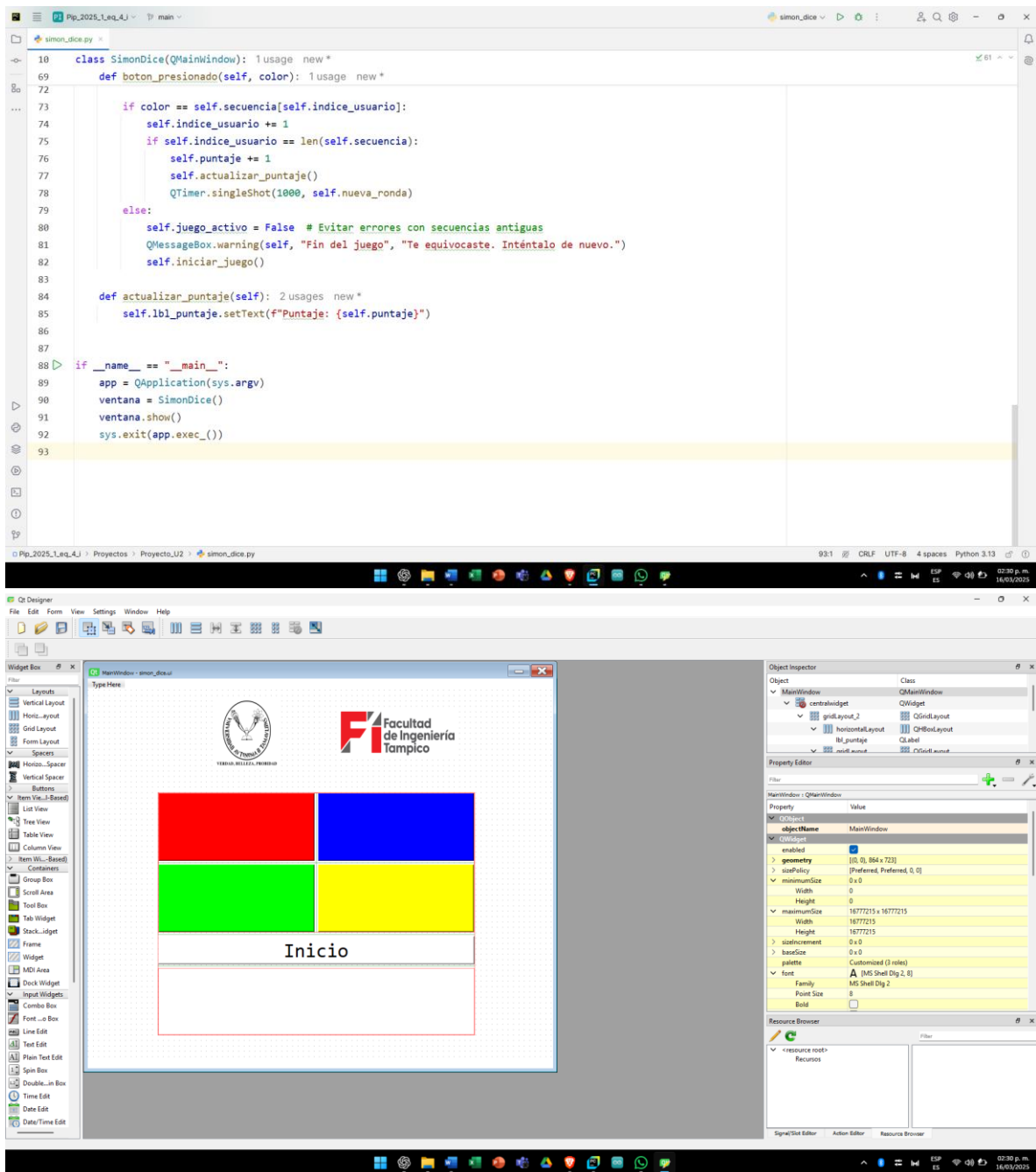
10 class SimonDice(QMainWindow): 1 usage new *
11
12 def cargar_logos(self): 1 usage new *
13     """Carga los logos en los QLabel de la interfaz."""
14     if hasattr(self, "lbl_logo_izquierdo") and hasattr(self, "lbl_logo_derecho"):
15         # Verifica si existen las etiquetas en el .ui
16         ruta_logo_izq = os.path.abspath("logo_izquierdo.png")
17         ruta_logo_der = os.path.abspath("logo_derecho.png")
18
19         if os.path.exists(ruta_logo_izq):
20             self.lbl_logo_izquierdo.setPixmap(QPixmap(ruta_logo_izq))
21             self.lbl_logo_izquierdo.setScaledContents(True)
22
23         if os.path.exists(ruta_logo_der):
24             self.lbl_logo_derecho.setPixmap(QPixmap(ruta_logo_der))
25             self.lbl_logo_derecho.setScaledContents(True)
26
27 def iniciar_juego(self): 2 usages new *
28     self.secuencia.clear()
29     self.puntaje = 0
30     self.indice_usuario = 0
31     self.juego_activo = True # Activar juego
32     self.actualizar_puntaje()
33     self.nueva_ronda()
34
35 def nueva_ronda(self): 2 usages new *
36     if not self.juego_activo:
37         return # Evitar que la secuencia siga ejecutándose tras reiniciar
38     self.indice_usuario = 0

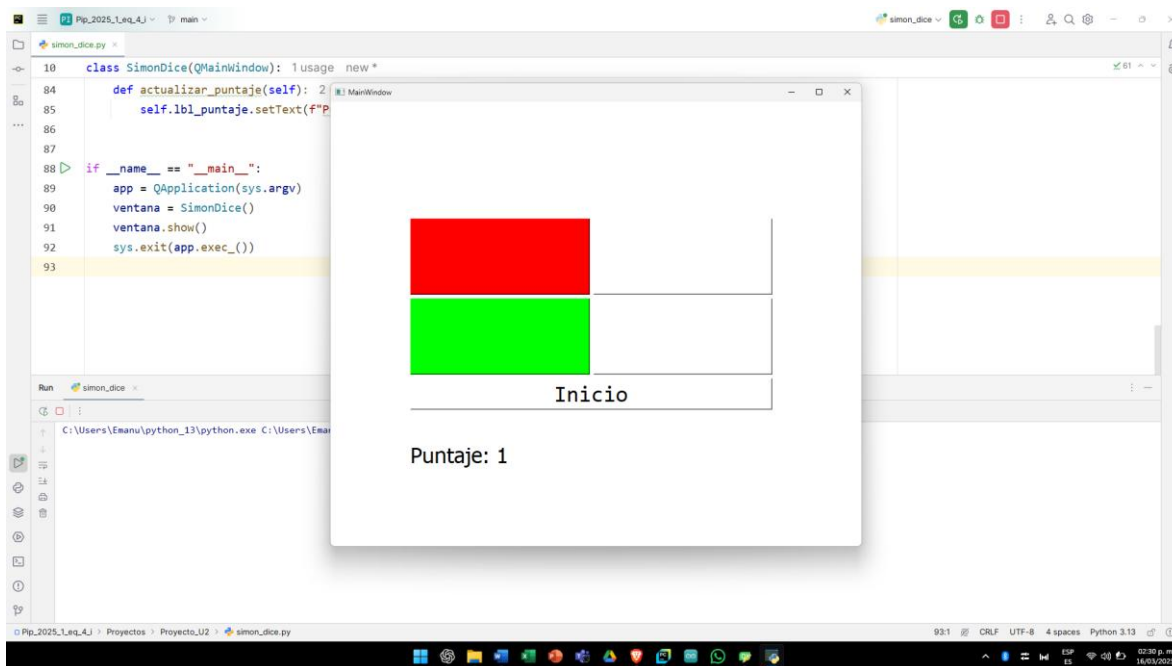
```

```

60 def mostrar_secuencia(self): 1 usage new *
61     for i, color in enumerate(self.secuencia):
62         QTimer.singleShot(i * 1000, lambda c=color: self.iluminar_botón(c))
63
64 def iluminar_botón(self, color): 1 usage new *
65     boton = self.colores[color]
66     boton.setStyleSheet("background-color: white;")
67     QTimer.singleShot(500, lambda: boton.setStyleSheet(""))
68
69 def botón_presionado(self, color): 1 usage new *
70     if not self.juego_activo:
71         return # Evitar interacciones si el juego ha sido reiniciado
72
73     if color == self.secuencia[self.indice_usuario]:
74         self.indice_usuario += 1
75         if self.indice_usuario == len(self.secuencia):
76             self.puntaje += 1
77             self.actualizar_puntaje()
78             QTimer.singleShot(1000, self.nueva_ronda)
79     else:
80         self.juego_activo = False # Evitar errores con secuencias antiguas
81         QMessageBox.warning(self, "Fin del juego", "Te equivocaste. Intentalo de nuevo.")
82         self.iniciar_juego()
83
84 def actualizar_puntaje(self): 2 usages new *
85     self.lbl_puntaje.setText(f"Puntaje: {self.puntaje}")
86

```



Conclusión de proyecto U2

El desarrollo del juego "Simón Dice" en **Python** con la ayuda de **Qt Designer** ha sido un proyecto que ha permitido adquirir valiosas experiencias tanto en programación como en diseño de interfaces gráficas y la creación de juegos interactivos. A lo largo de este proceso, se logró implementar un juego entretenido y desafiante que combina elementos visuales y sonoros para mejorar la experiencia del usuario.

La herramienta **Qt Designer** fue clave para crear una interfaz gráfica clara y fácil de usar, manteniendo el archivo en formato .ui para una mayor flexibilidad en el diseño. La lógica del juego, implementada en **Python**, permitió generar secuencias de colores de manera aleatoria y manejar los tiempos de cada secuencia mediante **QTimer**, lo que le dio dinamismo al juego.

Además, se incorporaron elementos interactivos como eventos de clic para la selección de colores y efectos visuales y sonoros para que el jugador se sintiera inmerso en la experiencia. El sistema de puntuación y la dificultad progresiva hicieron que el juego se volviera más interesante y retador a medida que avanzaba.

En resumen, este proyecto no solo proporcionó una forma divertida de aprender y aplicar conceptos técnicos de programación, sino que también permitió crear una experiencia interactiva y educativa. Los conocimientos adquiridos durante el desarrollo de este juego servirán como base para futuros proyectos que involucren interfaces gráficas y dinámicas de juegos en **Python**.